

BÀI 4: KIỂM THỬ GIÁ TRỊ BIÊN

Mục tiêu:

- **Về kiến thức:** cung cấp cho sinh viên kiến thức tổng quan về kiểm thử chức năng, kiến thức về kỹ thuật kiểm thử giá trị biên: phân loại kiểm thử giá trị biên, các bước thiết kế testcase
- **Về kỹ năng:** giúp sinh viên áp dụng kỹ thuật kiểm thử giá trị biên thiết kế được hệ thống testcase cho bài toán cụ thể

4.1 Tổng quan

Kiểm thử hàm (functional testing) là các hoạt động kiểm tra chương trình dựa trên tài liệu mô tả chức năng, yêu cầu phần mềm, hay còn gọi là *đặc tả chức năng* (functional specification). Đặc tả chức năng là tài liệu mô tả yêu cầu, hành vi của chương trình mong muốn. Tài liệu này là cơ sở để chúng ta tiến hành xây dựng các ca kiểm thử, thuật ngữ chuyên môn gọi là *thiết kế kiểm thử* (test design) và người thực hiện việc này là người thiết kế kiểm thử. Thiết kế kiểm thử là hoạt động chính trong kiểm thử hàm.

Cần nhấn mạnh rằng kiểm thử hàm là việc thiết kế các ca kiểm thử hàm của chương trình chỉ dựa trên đặc tả của chương trình mà không dựa trên việc phân tích mã nguồn của chương trình. Kiểm thử hàm còn được gọi với tên khác chính xác hơn là kiểm thử dựa trên đặc tả, hay kiểm thử hộp đen, hay kiểm thử chức năng. Còn việc thiết kế kiểm thử có dựa vào mã nguồn của chương trình được gọi là kiểm thử cấu trúc, hay kiểm thử hộp trắng.

Kiểm thử hàm là một kỹ thuật cơ bản giúp phát hiện nhiều lỗi mà kiểm thử hộp trắng không phát hiện được. Ví dụ dễ thấy nhất là các lỗi do cài đặt (implementation) không đầy đủ so với đặc tả. Nếu phần mềm thiếu hẳn một chức năng thì chúng ta dễ nhận thấy, nhưng nếu phần mềm thiếu một trường hợp của một chức năng thì việc này không dễ phát hiện nếu không kiểm tra kỹ chương trình so với đặc tả.

Tài liệu đặc tả dùng để thiết kế các ca kiểm thử có thể đơn giản là mô tả về chức năng của chương trình viết bằng ngôn ngữ của người sử dụng, nhưng thường là cả các tài liệu phân tích, thiết kế ở các mức độ chi tiết hơn với các hình vẽ, bảng

biểu, sơ đồ làm rõ hành vi của chương trình vì chúng mô tả sát hơn các tính chất hàm của chương trình mà chúng ta sẽ dựa vào đó để thiết kế kiểm thử. Chúng ta gọi chung các tài liệu này là đặc tả chức năng.

Dễ nhận thấy do chỉ dựa trên đặc tả chức năng, kiểm thử hàm có ưu điểm là có thể thực hiện được sớm, trước khi cài đặt chương trình. Thời điểm tốt để chúng ta bắt đầu thiết kế kiểm thử là khi tài liệu đặc tả đã ổn định, nhưng cũng có nhiều qui trình phần mềm hiện đại đang đẩy việc này lên sớm hơn, ngay trong giai đoạn xác định yêu cầu đã lồng ghép xác định các ca kiểm thử. Các ca kiểm thử này thường được mô tả dưới dạng các ví dụ, vừa có tác dụng giải thích, làm rõ tài liệu đặc tả, vừa có tác dụng sử dụng luôn làm các kịch bản kiểm thử, các ca kiểm thử.

Thông thường kết quả của việc thiết kế kiểm thử hàm sẽ đưa ra *đặc tả ca kiểm thử*. Tương tự như khái niệm đặc tả chương trình là mô tả chương trình mong muốn, đặc tả kiểm thử mô tả các ca kiểm thử cần được tạo ra. Từ một đặc tả ca kiểm thử chúng ta có thể tạo ra nhiều ca kiểm thử cụ thể để tiến hành kiểm tra với chương trình. Ví dụ với một hàm giải phương trình bậc hai với đầu vào là ba hệ số là a, b, c , đặc tả ca kiểm thử cho nghiệm kép là với mọi a, b, c thỏa mãn $b^2 = 4ac$. Từ đặc tả này ta có thể tạo ra các ca kiểm thử cụ thể, ví dụ $a = 1, b = 2, c = 1$, và kết quả mong đợi là nghiệm kép -1 .

Bản chất của các đặc tả ca kiểm thử là mỗi đặc tả ca kiểm thử sẽ xác định một lớp hành vi của chương trình. Do đó việc thiết kế kiểm thử hàm là việc phân tách đặc tả các hành vi có thể của chương trình thành một số lớp mà mỗi lớp này sẽ có hành vi nhất quán hay tương tự. Với ví dụ hàm giải phương trình bậc hai trên chúng ta có thể chia đặc tả thành hành vi phương trình cho nghiệm kép, hành vi phương trình cho kết quả vô nghiệm, v.v.

Việc phân tích đặc tả sẽ giúp chúng ta xác định và phân tách được các lớp này. Bên cạnh đó việc phân tích cũng thường giúp chúng ta xác định những điểm không rõ ràng, không đầy đủ của đặc tả. Đây là quá trình phân tích tổng hợp nhiều nguồn thông tin, từ đặc tả, từ kiến thức, kinh nghiệm của người thiết kế kiểm thử, do đó nó mang tính chủ công và cũng dễ mắc lỗi. Nhiều chuyên gia về thiết kế kiểm thử hàm nhiều khi vẫn bỏ sót một số ca kiểm thử quan trọng. Để giảm thiểu các sai sót này, chúng ta cần có phương pháp hệ thống giúp chia quá trình thiết kế này thành các

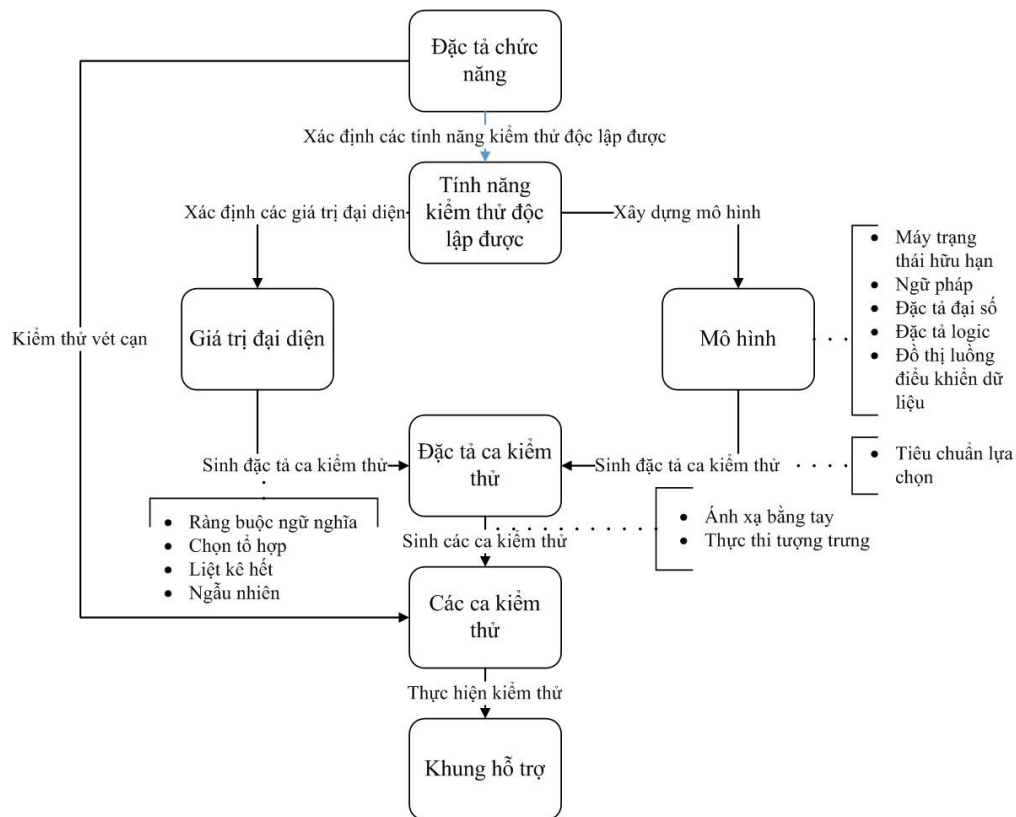
bước cơ bản để thực hiện. Từ đó một số bước sẽ được tự động hóa để giảm công sức, giảm sai sót, và tăng được chất lượng công việc. Thực tế việc tự động hoàn toàn chưa thực hiện được và một số bước cơ bản vẫn phải thực hiện thủ công. Do đó kỹ năng và kinh nghiệm của người thiết kế kiểm thử đóng vai trò quyết định đến chất lượng của bộ kiểm thử.

Để hiểu rõ hơn các công việc trong thiết kế kiểm thử hàm, chúng ta giả sử đang thiết kế kiểm thử cho chương trình P và giả sử $Y = P(X)$ trong đó X là vec-tơ của n biến đầu vào và Y là vec-tơ của m biến đầu ra. Dựa trên hiểu biết của chúng ta về đặc tả của chương trình P , với một vec-tơ đầu vào $X = (a_1, \dots, a_n)$ với a_i là các giá trị cụ thể thì chúng ta tính được một vec-tơ kết quả cụ thể $Y = (b_1, \dots, b_m)$. Kết quả Y này là được gọi là *kết quả mong đợi*. Kết quả thực của chương trình P với đầu vào X khi chạy chương trình là $P(X)$. Chúng ta cần kiểm tra kết quả mong đợi đúng bằng kết quả thực của chương trình, tức là Y và $P(x)$ là một hay không. Nếu đúng thì chương trình đã chạy đúng với X . Nếu chương trình chạy đúng với mọi giá trị X như mô tả trong đặc tả thì chúng ta kết luận chương trình P đã được cài đặt đúng so với đặc tả.

4.1.1 Phương pháp hệ thống

Việc xác định các ca kiểm thử từ đặc tả chức năng là quá trình phân tích để xác định và chia không gian đầu vào/đầu ra/hành vi của chương trình thành các miền con. Phương pháp hệ thống sẽ chia quá trình này thành các bước cơ bản, từ đó làm từng bước được dễ dàng hơn hoặc tự động hóa một số bước nếu có thể. Phương pháp hệ thống giúp chúng ta xác định bộ ca kiểm thử đã đủ chưa đồng thời tránh tạo ra nhiều ca kiểm thử trùng lặp, không cần thiết.

Hình 4.1 mô tả các bước chính của một phương pháp hệ thống. Một số bước trong qui trình này có thể phức tạp, một số bước khá đơn giản và một số bước có thể bỏ qua tùy theo miền ứng dụng và đặc tả chương trình cũng như kinh nghiệm của người thiết kế. Dựa trên qui trình tổng quát này chúng ta có thể đưa ra các qui trình cụ thể bằng việc cụ thể hóa các bước trong qui trình. Chúng ta sẽ xem xét chi tiết hơn các bước trong phương pháp này.



Hình 4.1: Các bước chính của phương pháp hệ thống cho kiểm thử hàm

Xác định các chức năng kiểm thử độc lập được: Đặc tả phần mềm trên thực tế là lớn và phức tạp ngay cả với những hệ thống nhỏ. Chúng thường được phân rã thành các đơn vị nhỏ hơn như các hệ thống con hay cụ thể hơn là các ca sử dụng (use case) hoặc các kịch bản sử dụng (user stories). Chúng ta gọi chung là các chức năng. Ví dụ trang web tìm kiếm của Google có thể có tính năng tìm kiếm, hiển thị kết quả, hiển thị quảng cáo, hay đăng ký người sử dụng mới, v.v.. Tuy nhiên ranh giới của một chức năng nhiều khi không rõ ràng. Ví dụ chức năng tìm kiếm của Google Search có chức năng tính toán một biểu thức số học nếu văn bản tìm kiếm là một biểu thức số học.

Khi thiết kế kiểm thử chúng ta tách các chức năng càng chi tiết, cụ thể càng tốt vì nó vừa giúp đơn giản việc thiết kế kiểm thử và vừa cho phép chúng ta xác định lỗi trong chương trình dễ dàng hơn. Mỗi ca kiểm thử chỉ nên kiểm tra một kịch bản cụ thể, chi tiết của một chức năng của hệ thống.

Tiếp đó với mỗi chức năng chúng ta sẽ xác định tất cả các đầu vào có ảnh hưởng đến việc thực hiện chức năng đó và các đầu ra có thể bị tác động bởi chức năng đó. Các đầu vào và đầu ra thường được mô tả tường minh trong tài liệu đặc tả. Nhưng

cũng có nhiều trường hợp các đầu vào và đầu ra này nằm ẩn trong tài liệu đặc tả mà người thiết kế phải phát hiện ra. Ví dụ như các đặc tả không hình thức mô tả chức năng đăng ký tài khoản mới thường chỉ nêu các trường dữ liệu cần khai báo nhưng không nói gì đến chi tiết kiểu dữ liệu sẽ lưu thông tin hồ sơ người sử dụng. Hay một ví dụ khác khá rắc rối với tên tiếng Việt khi chúng ta chỉ có hai trường họ và tên để nhập vào. Một là thông thường chúng ta không nói ra cụ thể chiều dài tối đa, tối thiểu, hay qui định bằng mã ký tự của các trường này. Hai là chúng ta không nói rõ phần tên đệm sẽ nằm ở phần họ hay phần tên, nên khi người sử dụng nhập dữ liệu sẽ xảy ra tình trạng mỗi người nhập một kiểu. Tên là Nguyễn Văn Hùng thì có người nhập phần họ là Nguyễn, phần tên là Văn Hùng; có người lại nhập họ là Nguyễn Văn, tên là Hùng; lại có người chỉ nhập họ là Nguyễn, tên là Hùng, vì nghĩ phần Văn là tên đệm không hỏi thì không nhập.

Xác định các lớp giá trị đại diện: Các giá trị đại diện có thể được xác định trực tiếp từ đặc tả phi hình thức, như ngôn ngữ tự nhiên. Chúng cũng có thể được xác định gián tiếp dựa trên một mô hình được xây dựng để phục vụ cho việc này. Trong cả hai trường hợp này mục tiêu đều là xác định các giá trị của mỗi đầu vào một cách độc lập, bằng việc lấy các giá trị đã liệt kê tường minh trong đặc tả, hoặc thông qua một mô hình phù hợp. Chúng ta chưa xét đến tổ hợp của các giá trị này. Lý do là chúng ta đang tách bài toán xác định các lớp giá trị của mỗi đầu vào và bài toán tổ hợp chúng để tạo thành các ca kiểm thử. Tức là chúng ta đang tách một bước phức tạp thành các bước đơn giản hơn.

Với các đặc tả không hình thức người thiết kế kiểm thử phải dựa vào việc liệt kê tường minh các lớp giá trị đại diện. Trong trường hợp này người thiết kế kiểm thử cần cẩn thận xét tất cả các trường hợp có thể và khai thác tối đa những thông tin có sẵn trong đặc tả. Tương tự, với kết quả mong đợi đầu ra, chúng ta cũng có thể xác định các lớp giá trị của miền đầu ra, cũng như các giá trị biên và các giá trị đặc biệt để có lỗi.

Ví dụ khi xem xét các thao tác trên một danh sách không rỗng chúng ta cần xét cả trường hợp danh sách rỗng (giá trị lỗi) và danh sách có một phần tử (giá trị biên chiều dài danh sách) là các trường hợp đặc biệt. Ở bước này chúng ta chỉ quan tâm đến tính chất của giá trị, chứ chưa cần cụ thể giá trị là gì. Tức là chúng ta quan tâm đến đặc tả ca kiểm thử chứ chưa phải ca kiểm thử cụ thể. Với ví dụ danh sách một

phần tử, chúng ta chưa cần quan tâm phần tử của danh sách này cụ thể là gì, chỉ biết danh sách chỉ có một phần tử.

Để liệt kê các giá trị không tường minh chúng ta cần xây dựng mô hình hoặc một phần của mô hình của đặc tả. Ví dụ chúng ta có thể dùng văn phạm để mô tả ngôn ngữ và sinh các từ của ngôn ngữ này để liệt kê các giá trị cần xác định. Mô hình này có thể có sẵn trong đặc tả nhưng thông thường người thiết kế kiểm thử phải xây dựng chúng.

Trong hai cách trên, việc liệt kê trực tiếp có vẻ đơn giản hơn và ít tốn kém hơn việc xây dựng mô hình để sinh ra các giá trị. Tuy nhiên về lâu về dài các mô hình sẽ có giá trị hơn, khi khối lượng giá trị lớn chúng ta có thể tự động hóa việc liệt kê, hay chúng ta cũng có thể điều chỉnh số lượng ca kiểm thử nhiều hay ít để cân bằng với các ràng buộc kiểm thử khác, như chi phí, thời gian. Các mô hình cũng có thể được sử dụng lại, sửa đổi dễ dàng hơn trong quá trình phát triển hệ thống. Quyết định cuối cùng nên dùng cách nào vẫn tùy thuộc vào lĩnh vực ứng dụng, kỹ năng của người thiết kế kiểm thử, và các công cụ thích hợp đang có.

Sinh các đặc tả ca kiểm thử: Đặc tả kiểm thử được tạo ra bằng việc tổ hợp các giá trị của các đầu vào của chương trình đang được kiểm thử. Bước trên chúng ta đã xác định các giá trị đại diện thì ở bước này chúng ta sinh các tổ hợp của chúng bằng tích Đề-các của các giá trị đại diện đã chọn. Nếu ở bước trước chúng ta tạo ra mô hình hình thức thì ở bước này các đặc tả ca kiểm thử sẽ là các hành vi cụ thể hoặc tổ hợp của các tham số của mô hình và một đặc tả ca kiểm thử có thể được thỏa mãn bởi nhiều đầu vào cụ thể. Trong cả hai cách này việc tạo ra tất cả các tổ hợp thường tạo ra số lượng quá lớn các ca kiểm thử.

Ví dụ một hàm có năm đầu vào, mỗi đầu vào có sáu giá trị đại diện thì tích Đề-các sẽ tạo ra $6^5 = 7,776$ đặc tả ca kiểm thử. Số lượng này là quá lớn với một chức năng, ngay cả khi được tự động hóa và chạy bằng máy tính hoàn toàn. Thông thường, rất nhiều tổ hợp là không hợp lệ hoặc không khả thi. Chúng ta sẽ học các phương pháp để giảm số tổ hợp này nhưng vẫn đảm bảo chất lượng của bộ kiểm thử.

Để giảm số ca kiểm thử, chúng ta cần loại bỏ các tổ hợp giá trị không hợp lệ bằng việc đưa thêm các ràng buộc về cách tổ hợp. Ví dụ hàm *NextDate* chúng ta cần đưa ràng buộc để tránh tạo ra các giá trị không hợp lệ như ngày 31/11/2013. Hoặc chúng

ta cũng không xét hết tất cả các tích Đề-các mà chỉ cần mỗi cặp giá trị của hai biến bất kỳ đều xuất hiện là đủ. Quá trình này có thể không phải một bước làm là xong mà có thể cần điều chỉnh và thử một số lần để có kết quả là bộ ca kiểm thử phù hợp, vừa hạn chế được các ca kiểm thử vô lý không cần thiết, vừa tiết kiệm được chi phí thực hiện kiểm thử.

Sinh ca kiểm thử và thực hiện kiểm thử: Quá trình kiểm thử hoàn tất khi chúng ta cụ thể hóa các đặc tả kiểm thử thành các ca kiểm thử cụ thể, rồi tính kết quả mong đợi của từng ca kiểm thử, và tiến hành kiểm tra với phần mềm. Việc thực hiện kiểm thử cần được tự động hóa tối đa khi có thể, vì một hệ thống phần mềm cần được kiểm thử rất nhiều lần. Ngay cả khi sản phẩm hoàn tất, trong tương lai chúng ta vẫn sẽ cần thực hiện kiểm thử nhiều lần nữa, khi phần mềm tiến hóa, thay đổi theo những yêu cầu mới, hay các lỗi được phát hiện và phần mềm cần phải sửa.

4.1.2 Lựa chọn phương pháp phù hợp

Trong các chương sau chúng ta sẽ thấy một số kỹ thuật kiểm thử hàm thích hợp cho các loại đặc tả chương trình khác nhau. Với một đặc tả có thể có một số kỹ thuật phù hợp để xây dựng các ca kiểm thử và một số kỹ thuật sẽ rất khó áp dụng hoặc kết quả sẽ không thỏa mãn. Một số kỹ thuật có thể cho kết quả như nhau với một số đặc tả. Một số kỹ thuật lại bổ sung cho nhau nên cần kết hợp chúng. Ví dụ như khi mỗi kỹ thuật lại áp dụng tốt cho những khía cạnh khác nhau của cùng một đặc tả, hoặc ở các giai đoạn khác nhau của việc sinh ca kiểm thử.

Việc lựa chọn kỹ thuật để tạo ra các ca kiểm thử phụ thuộc vào các yếu tố: tính tự nhiên của đặc tả, dạng của đặc tả, kinh nghiệm của người thiết kế kiểm thử, cấu trúc của tổ chức, công cụ sẵn có, ngân sách và các ràng buộc về chất lượng, chi phí thiết kế và cài đặt các hệ thống hỗ trợ.

Tính tự nhiên và dạng của đặc tả: Các kỹ thuật kiểm thử khai thác các đặc điểm khác nhau của đặc tả. Ví dụ, khi biên đầu vào có một số ràng buộc thì có thể kỹ thuật kiểm thử lớp tương đương sẽ phù hợp. Nếu có sự chuyển dịch trong các trạng thái của hệ thống thì phương pháp kiểm thử dựa trên máy trạng thái có thể nên áp dụng. Hay khi các đầu vào thay đổi với kích thước không xác định được thì phương pháp dùng ngữ pháp có thể phù hợp.

Kinh nghiệm của người thiết kế kiểm thử: Kinh nghiệm của người kiểm thử

hiều khi sẽ ảnh hưởng đến kỹ thuật kiểm thử nào được sử dụng. Ví dụ một người thiết kế kiểm thử giỏi về bảng quyết định sẽ thích kỹ thuật đó khi một kỹ thuật khác cũng có thể áp dụng được.

Công cụ: Một số kỹ thuật đòi hỏi sử dụng các công cụ mà đôi khi phải mua nên chi phí cũng là một yếu tố ảnh hưởng đến việc lựa chọn kỹ thuật kiểm thử. Ví dụ đặc tả sử dụng UML và có công cụ thương mại sinh ca kiểm thử dựa trên một số biểu đồ UML thì công cụ đóng vai trò quan trọng ảnh hưởng đến phương pháp thực hiện kiểm thử, nếu ngân sách cho kiểm thử cho phép. Một số đơn vị làm phần mềm chuyên nghiệp đôi khi phải xây dựng các công cụ này để phục vụ cho đơn vị, hoặc đôi khi cho từng dự án cụ thể.

Ngân sách và ràng buộc chất lượng: Ngân sách và ràng buộc chất lượng có thể dẫn đến sự lựa chọn khác nhau về phương pháp kiểm thử. Ví dụ nếu yêu cầu là nhanh, kiểm thử tự động, nhưng không cần độ tin cậy cao thì kiểm thử ngẫu nhiên là phù hợp. Ngược lại, yêu cầu độ tin cậy phải rất cao thì chúng ta phải sử dụng các phương pháp tinh vi để kiểm thử một cách kỹ lưỡng nhất có thể. Khi chọn một phương pháp, điều quan trọng là phải đánh giá tất cả các yếu tố và chi phí liên quan.

Chi phí nền tảng kiểm thử: Mỗi đặc tả ca kiểm thử sẽ phải được cụ thể hóa thành một ca kiểm thử và được chạy đi chạy lại nhiều lần trong suốt vòng đời của sản phẩm. Mỗi lần chạy một ca kiểm thử bao gồm việc phải đưa đầu vào vào chương trình, thực hiện chương trình, và so sánh kết quả đầu ra thực tế với đầu ra mong đợi. Nếu chúng ta tự động hóa được tất cả các công việc này thì phương pháp tổ hợp sinh ra tất cả các tổ hợp có thể áp dụng. Tuy nhiên nếu một số bước phải thực hiện bằng tay, hoặc chúng ta phải viết mã cho từng đặc tả kiểm thử này thì có lẽ chúng ta phải chọn phương pháp khác để khả thi về mặt thời gian và công sức.

Tóm lại kiểm thử hàm cũng giống nhiều hoạt động kỹ thuật khác đòi hỏi phải phân tích kỹ lưỡng các ưu nhược điểm để chọn phương pháp cân bằng phù hợp với các khó khăn và các vấn đề không lường hết được mà phải là người thiết kế có kinh nghiệm mới có cảm giác tốt được. Kiểm thử hàm không phải là việc tập để chọn một phương pháp tối ưu mà là một loạt các hoạt động để tìm ra sự tổ hợp của một số mô hình và kỹ thuật để tạo ra một tập các ca kiểm thử thỏa mãn chi phí và ràng buộc chất lượng đặt ra. Sự cân bằng này cũng có thể mở rộng ra ngoài thiết kế kiểm

thử sang thiết kế phần mềm để thuận lợi cho việc kiểm thử. Thiết kế phần mềm thích hợp sẽ không chỉ giúp quá trình phát triển được tốt hơn mà còn giúp việc kiểm thử cũng thuận lợi để giúp tiết kiệm chi phí tổng thể của dự án phần mềm.

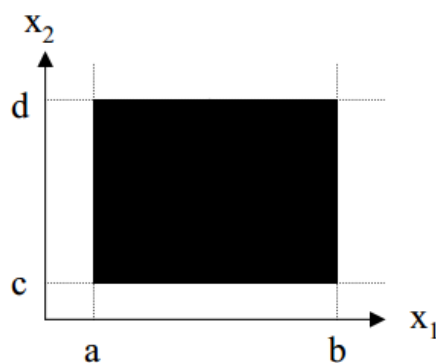
4.2 Kiểm thử giá trị biên

Kiểm thử giá trị biên (boundary testing) là một trong những kỹ thuật được áp dụng phổ biến nhất trong kiểm thử hàm. Chúng ta sẽ coi một chương trình là một hàm toán học với đầu vào của chương trình tương ứng với các tham số của hàm và đầu ra của chương trình là giá trị trả về của hàm. Vì hàm toán học là ánh xạ từ miền xác định của hàm đến miền giá trị của hàm. Chúng ta sẽ tập trung vào các giá trị biên và cạnh biên của hai miền đầu vào và đầu ra này của hàm để xây dựng các ca kiểm thử. Khi chúng ta dùng biên đầu ra tức là chúng ta cho các kết quả mong đợi nằm ở trên biên và cạnh biên đầu ra.

Giả sử $y = f(x_1, x_2)$ với $x_1, x_2, y \in \mathbb{N}$ là một hàm toán học của một chương trình. Khi đó thông thường x_1 và x_2 có miền xác định thể hiện bằng các biên. Ví dụ:

$$a \leq x_1 \leq b \text{ và } c \leq x_2 \leq d$$

trong đó a, b, c, d là các hằng số nào đó. Hình 4.2 thể hiện miền xác định của hai biến này.

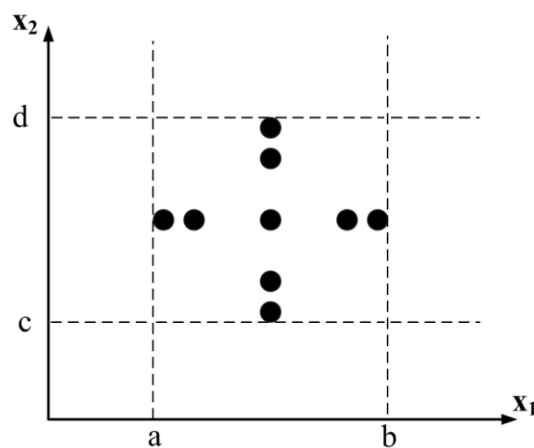


Hình 4.2: Miền xác định của hàm hai biến.

Ý tưởng kiểm thử giá trị biên xuất phát từ quan sát nhiều lỗi xảy với các giá trị biên này, khi chúng ta không kiểm tra khoảng giá trị hợp lệ của dữ liệu được nhập vào, hay do lỗi lập trình hoặc đặc tả làm các biểu thức điều kiện không chính xác. Ví dụ đúng ra phải là dấu $\leq$ nhưng người lập trình hoặc đặc tả lại chỉ viết $<$ hoặc ngược lại. Kiểm thử với các giá trị biên giúp chúng ta phát hiện các lỗi này. Trong nhiều trường hợp các biên này là ẩn, không được viết rõ ra trong yêu cầu nên lập

trình viên dễ mắc lỗi không kiểm tra các giá trị đầu vào, hoặc không kiểm tra kết quả có hợp lệ không trước khi trả về.

Để tăng khả năng phát hiện lỗi, kiểm thử giá trị biên thường lấy năm ca kiểm thử cho mỗi biến là các giá trị: cực đại, cực tiểu, các giá trị cạnh chúng trong miền xác định (gọi là cận biên hoặc cạnh biên), và một giá trị ở giữa miền xác định đại diện cho giá trị thông thường. Chúng ta đánh chỉ số cho chúng lần lượt là *max*, *min*, *min*, *max*⁺ và *nom*. Hình 4.3 minh họa các giá trị này.



Hình 4.3: Các ca kiểm thử phân tích giá trị biên cho một hàm hai biến.

Thông thường lỗi chương trình xảy ra ngay khi có một sai sót trong phần mềm, chứ không cần kết hợp của nhiều sai sót mới gây ra lỗi. Khi đó các ca kiểm thử theo phương pháp kiểm thử giá trị biên được xây dựng bằng cách lấy một bộ giá trị *nom* của các biến, rồi lần lượt thay mỗi giá trị đó của từng biến bằng giá trị biên và cận biên để tạo ra ca kiểm thử mới. Ví dụ với hàm *f* trên ta có bộ kiểm thử sau, thể hiện trên Hình 4.3.

$$\{(x_{1nom}, x_{2nom}), (x_{1min}, x_{2nom}), (x_{1min+}, x_{2nom}), (x_{1max-}, x_{2nom}), (x_{1max}, x_{2nom}), (x_{1nom}, x_{2min}), (x_{1nom}, x_{2min+}), (x_{1nom}, x_{2max-}), (x_{1nom}, x_{2max})\}$$

Tổng quát hóa kiểm thử giá trị biên cho hàm *n* biến số và mỗi biến có các giá trị biên và cận biên khác nhau ta có thể dễ thấy sẽ có $1 + 4n$ ca kiểm thử vì xuất phát từ một ca kiểm thử gồm các giá trị trung bình của các biến, ta thay nó bằng bốn giá trị biên và cận biên: *min*, *min*⁺, *max*, và *max*. Với *n* biến sẽ tạo thêm $4n$ bộ kiểm thử. Do đó tổng số ca kiểm thử là $1 + 4n$. Tuy nhiên tùy theo miền xác định của biến mà số

lượng này thực tế có thể ít hơn. Ví dụ biến nguyên thuộc khoảng [1, 2] thì không có cận biên và không có giá trị ở giữa.

Với cách tạo bộ kiểm thử giá trị biên này, mỗi giá trị biên và cận biên xuất hiện một lần với các giá trị trung bình của các biến còn lại, chứ không phải tổ hợp các bộ giá trị của các biến. Tổ hợp của các biến được gọi là kiểm thử giá trị biên mạnh sẽ được thảo luận trong phần tiếp theo.

Trong thực tế, miền xác định của các biến không chỉ là các miền số, nên chúng ta cần có các vận dụng thích hợp để xác định các giá trị biên tinh tế hơn. Ví dụ trong chương trình NextDate biến tháng month có thể là số nguyên trong khoảng từ 1 đến 12. Nhưng không phải ngôn ngữ lập trình nào cũng cho phép khai báo miền xác định của biến trong khoảng này. Chúng ta cũng có thể dùng kiểu liệt kê (enumeration) để khai báo các tháng bằng tên Januray, ..., December và chúng ta vẫn có giá trị biên và cận biên. Tuy nhiên như trong ngôn ngữ lập trình Java chúng ta phải dùng biến kiểu nguyên int và không khai báo miền giá trị cho chúng được, thì chúng ta sẽ phải dùng các cận của kiểu làm biên. Trong Java, biên của số nguyên là Integer.MIN_VALUE và Integer.MAX_VALUE. Trong C, chúng ta cũng có các khoảng giá trị này.

Ưu nhược điểm: Khi các biến của hàm là độc lập, không có quan hệ ràng buộc lẫn nhau, thì kiểm thử giá trị biên tỏ ra hiệu quả. Nhưng khi chúng có quan hệ phụ thuộc nào đó thì phương pháp này dễ tạo ra các ca kiểm thử không hợp lý, vì các giá trị cực đại, cực tiểu có thể không kết hợp với nhau để tạo thành ca kiểm thử hợp lệ.

Ví dụ khác với hàm NextDate thì biến ngày, tháng và năm có ràng buộc. Một số tháng có 31 ngày, một số lại có 30 ngày. Năm nhuận lại có ràng buộc với số ngày trong tháng hai. Nếu không xét sự phụ thuộc này kiểm thử giá trị biên không tạo ra được ca kiểm thử ngày 28 tháng 2 năm 2012, vì 28 không là biên và cận biên của khoảng ngày từ 1 đến 31.

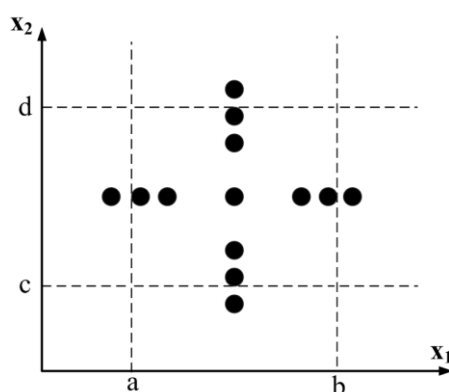
Kiểm thử giá trị biên cũng không quan tâm đến tính chất, đặc trưng của hàm, đồng thời cũng không xét đến ngữ nghĩa của biến hay quan hệ giữa các biến. Nó chỉ máy móc lấy các giá trị biên, cận biên và trung bình để tổ hợp tạo ra các ca kiểm thử.

Ưu điểm của phương pháp này là đơn giản và có thể tự động hóa việc sinh các ca kiểm thử khi chúng ta đã xác định được các biên của các biến. Với nhiều kiểu dữ

liệu cơ bản có sẵn chúng ta cũng có thể xây dựng sẵn các biên này để sử dụng.

4.2.1 Kiểm thử giá trị biên mạnh

Kiểm thử giá trị biên mạnh là mở rộng của kiểm thử giá trị biên bằng việc bổ sung các giá trị cận biên bên ngoài miền xác định. Ngoài 5 giá trị biên đã nêu ở phần trước chúng ta lấy thêm các giá trị cận biên ở ngoài miền xác định là $max+$ và min như trong Hình 4.4. Các ca kiểm thử này sẽ giúp ta kiểm tra chương trình với dữ liệu không hợp lệ, nằm ngoài khoảng mong đợi. Ví dụ khi nhập ngày 32/1/2013 chương trình cần có thông báo lỗi thích hợp.



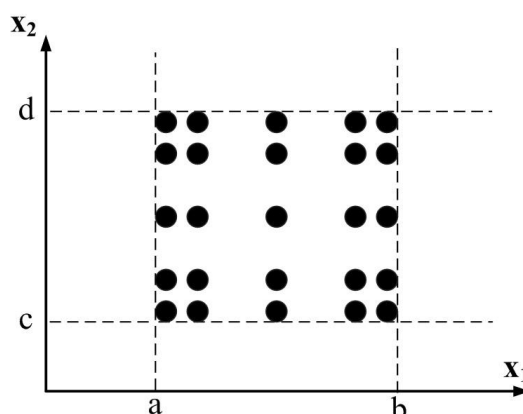
Hình 4.4: Các ca kiểm thử mạnh cho hàm hai biến.

Các ca kiểm thử mạnh này giúp chúng ta kiểm tra xem chương trình có xử lý các ngoại lệ hay kiểm tra biến đầu vào hay không. Với các ngôn ngữ lập trình không kiểm tra kiểu khi biên dịch hay dữ liệu được nhập vào bên ngoài, do người sử dụng đưa vào hoặc lấy từ hệ thống khác, việc này là rất cần thiết. Rất nhiều lập trình viên mới vào nghề thường xuyên không kiểm tra dữ liệu đầu vào nên chương trình chỉ chạy với dữ liệu nhập đúng như suy nghĩ của người lập trình. Hậu quả của việc này không chỉ đơn thuần là chương trình bị sai, mà thường gây vi phạm bộ nhớ, dẫn đến tắt chương trình (crash), và là một trong những lỗ hổng an ninh dễ bị khai thác.

4.2.2 Kiểm thử giá trị biên tổ hợp

Ở trên chúng ta chỉ tạo các ca kiểm thử với giá trị biên và cận biên cho từng biến. Nếu chúng ta mở rộng với hai hoặc với tất cả các biến đều được đẩy đến giá trị biên và cận biên thì chúng ta sẽ tạo ra được các ca kiểm thử giá trị biên tổ hợp. Tức là từ bộ giá trị 5 phần tử min , $min+$, nom , max và $max+$ của mỗi biến ta lấy tích Đề-Các (Cartesian) của chúng để tạo ra các ca kiểm thử. Với hai biến Hình 4.5 minh họa

các ca kiểm thử này.



Hình 4.5: Các ca kiểm thử biên tổ hợp của hàm hai biến.

Có thể thấy cách tổ hợp các biên và cận biên này sẽ kiểm tra kỹ hơn kiểm thử giá trị biên thông thường. Tuy nhiên số ca kiểm thử theo cách này tăng lên đáng kể, lên đến 5^n ca kiểm thử so với $4n + 1$ ca theo kiểm thử biên thông thường.

Tương tự như kiểm thử giá trị biên mạnh, ta có thể mở rộng kiểm thử biên tổ hợp với bộ 7 giá trị của kiểm thử giá trị biên mạnh. Chúng ta sẽ kiểm tra được kỹ hơn nhưng cũng mất nhiều công sức hơn, đến 7^n ca kiểm thử.

4.2.3 Kiểm thử các giá trị đặc biệt

Kiểm thử các giá trị đặc biệt cũng là một phương pháp phổ biến. Đây cũng là phương pháp trực quan nhất và không theo một khuôn dạng cụ thể nào. Dựa trên hiểu biết về bài toán và miền ứng dụng kết hợp với kinh nghiệm cá nhân, người kiểm thử đưa ra các giá trị kiểm thử. Do đó không có hướng dẫn cụ thể nào cho phương pháp này. Mức độ hiệu quả của phương pháp này phụ thuộc nhiều vào khả năng của người kiểm thử. Trên thực tế các đơn vị phát triển phần mềm vẫn áp dụng phương pháp này, vì nhiều khi nó giúp phát hiện lỗi nhanh, không tốn nhiều công sức.

Ví dụ dựa trên hiểu biết về số ngày của các tháng trong một năm chúng ta sẽ kiểm thử các ngày 28 tháng 2 và 29 tháng 2 ở cả năm nhuận và năm thường. Số lượng ca kiểm thử sẽ ít hơn nhiều so với kiểm thử giá trị biên và cũng hiệu quả trong việc kiểm tra lỗi. Đây cũng là tính chất “thủ công” của nghề kiểm thử phần mềm – “khéo tay” và có kinh nghiệm sẽ làm tốt hơn.

❖ Kinh nghiệm áp dụng

Kiểm thử giá trị biên là phương pháp “thô” nhất trong các phương pháp kiểm thử hàm. Phương pháp này nên áp dụng cho các hàm có các biến đầu vào độc lập, không phụ thuộc vào nhau. Khi chúng ta có giả định một khiếm khuyết trong chương trình sẽ gây ra lỗi ngay thì chúng ta sử dụng kiểm thử biên thông thường. Trái lại khi lỗi thường xuất hiện khi có hơn một khiếm khuyết trong chương trình thì chúng ta cần sử dụng kiểm thử giá trị biên mạnh. Các phương pháp này đều đơn giản, dễ dàng tự động hóa việc sinh và chạy các ca kiểm thử này.

Phần này chúng ta chủ yếu tập trung vào biên của biến đầu vào. Chúng ta có thể mở rộng cho biến đầu ra, tức là kết quả của hàm, sao cho các kết quả này có thể nằm ở biên, cận biên. Ngoài ra trong chương trình máy tính còn có các biến cục bộ, hay các biến toàn cục khác. Chúng ta có thể vận dụng mở rộng cho các biến này để có thể phát hiện thêm các lỗi tinh vi khác trong chương trình.

Câu hỏi cuối bài:

1. Các bước thực hiện kiểm thử giá trị biên
2. Phân loại kiểm thử giá trị biên
3. Kinh nghiệm áp dụng kiểm thử giá trị biên
4. Các bài thực hành:

Bài 1: Số thứ tự trên hệ thống điều khiển cổ phiếu có thể sắp xếp từ 10000 đến 99999 (bao gồm cả 10000 và 99999).

Yêu cầu: Hãy chỉ ra các đầu vào nào có thể là một kết quả của thiết kế kiểm thử chỉ cho các lớp tương đương đúng và các lớp biên đúng?

Bài 2: Một hệ thống được thiết kế để đóng thuế như sau: “Một nhân viên có lương £4000 thì được miễn thuế. £1500 tiếp theo bị đánh thuế 10%. £28000 tiếp theo bị đánh thuế 22%. Bất kỳ lượng tiền lớn hơn sau đó đều bị đánh thuế 44%”.

Yêu cầu: Giá trị nào là một trường hợp phân tích giá trị biên đúng cho giá trị tiền lương thấp nhất bị đánh phần trăm thuế lớn nhất?

Bài 3: Một hệ thống quản lý kho hàng yêu cầu nhập số lượng sản phẩm, với điều kiện:

- Số lượng sản phẩm phải nằm trong khoảng **10 đến 1000**.

Yêu cầu:

- (a) Xác định các giá trị biên cần kiểm thử.
- (b) Viết kịch bản kiểm thử cho các giá trị đó

Bài 4: Một ứng dụng ngân hàng yêu cầu khách hàng nhập số tiền rút, với điều kiện:

- Số tiền tối thiểu: **100**.
- Số tiền tối đa: **5000**.
- Số tiền phải là bội số của **50**.

Yêu cầu:

- (a) Xác định giá trị biên cho từng điều kiện trên.
- (b) Tạo các test case với các giá trị này

Bài 5: Một ứng dụng đặt vé máy bay yêu cầu nhập các thông tin sau:

- Số vé đặt: **1 đến 5 vé**.
- Tổng số kg hành lý miễn phí: **0 đến 30 kg**.
- Số tuổi của hành khách: **0 đến 120 tuổi**.

Yêu cầu:

- (a) Xác định giá trị biên cần kiểm thử cho mỗi điều kiện.
- (b) Viết các test case tương ứng, đảm bảo bao quát cả giá trị biên.